

““latex

1 Implementation Order

Target CPU:

- RV64GC
- Multicore
- Out-of-order execution
- Linux-capable
- M/S/U privilege modes
- Sv39 virtual memory
- CLINT + PLIC interrupts
- L1/L2 cache hierarchy
- MESI cache coherence

1.1 Implementation Order

1. `riscv_types_pkg.sv`

Define the shared RISC-V types, enums, instruction formats, opcodes, CSR addresses, exception causes, interrupt causes, immediate types, ALU operations, load/store operations, branch operations, and decoded instruction bundle.

2. `physical_register_file.sv`

Implement the integer physical register file used by register renaming and out-of-order execution. Register zero must always read as zero.

3. `immediate_generator.sv`

Generate I-type, S-type, B-type, U-type, J-type, and CSR immediates from a 32-bit RISC-V instruction.

4. `integer_alu.sv`

Implement RV64 integer arithmetic, logic, shift, compare, and word operations.

5. `decoder.sv`

Decode RV64GC 32-bit instructions into control signals. Compressed instructions are expanded before entering this decoder.

6. `compressed_decoder.sv`

Decode RVC 16-bit compressed instructions and expand them into normal 32-bit RISC-V instructions.

7. `branch_unit.sv`
Resolve conditional branches, JAL, and JALR. Generate branch-taken results and redirect targets.
8. `multiplier_divider.sv`
Implement the M extension: MUL, MULH, MULHSU, MULHU, DIV, DIVU, REM, REMU, and RV64 word variants.
9. `floating_point_unit.sv`
Implement the F and D extensions for single-precision and double-precision floating-point operations.
10. `csr_unit.sv`
Implement Zicsr CSR read/write/set/clear instructions and machine, supervisor, and user-visible CSR state.
11. `trap_controller.sv`
Handle exceptions, interrupts, ECALL, EBREAK, MRET, SRET, and precise trap redirection.
12. `privilege_controller.sv`
Track current privilege mode and enforce M/S/U privilege rules.
13. `pc_unit.sv`
Generate next PC from sequential execution, branch prediction, branch redirect, trap redirect, return from trap, and reset vector.
14. `fetch_queue.sv`
Buffer fetched instructions between frontend and decode.
15. `btb.sv`
Implement the branch target buffer.
16. `bht_pht.sv`
Implement branch direction prediction.
17. `ras.sv`
Implement return address stack for call/return prediction.
18. `branch_predictor.sv`
Integrate BTB, BHT/PHT, and RAS into the frontend predictor.
19. `register_alias_table.sv`
Map architectural registers to physical registers.
20. `free_list.sv`
Track free physical registers.

21. `rename_checkpoint.sv`
Save rename state for branch misprediction recovery.
22. `reorder_buffer.sv`
Track in-flight instructions and enforce in-order commit.
23. `reservation_station.sv`
Hold decoded and renamed instructions until operands are ready.
24. `issue_logic.sv`
Select ready instructions and issue them to execution units.
25. `common_data_bus.sv`
Broadcast execution results to the ROB, reservation stations, and physical register file.
26. `wakeup_logic.sv`
Wake dependent instructions when their source physical registers become ready.
27. `rob_commit_logic.sv`
Commit completed instructions in program order and handle precise exceptions.
28. `address_generation_unit.sv`
Compute load/store virtual addresses.
29. `load_store_queue.sv`
Track in-flight loads and stores and enforce memory ordering.
30. `store_buffer.sv`
Hold committed stores before they write into cache or memory.
31. `load_store_unit.sv`
Execute LB, LH, LW, LD, LBU, LHU, LWU, SB, SH, SW, and SD.
32. `atomic_memory_unit.sv`
Implement LR/SC and AMO instructions from the A extension.
33. `memory_ordering_unit.sv`
Enforce RVWMO ordering rules, FENCE, FENCE.I, and atomic ordering.
34. `itlb.sv`
Translate instruction virtual addresses.
35. `dtlb.sv`
Translate load/store virtual addresses.
36. `page_table_walker.sv`
Walk Sv39 page tables on TLB misses.

37. `sfence_vma_unit.sv`
Flush address-translation state when software executes SFENCE.VMA.
38. `mmu.sv`
Integrate ITLB, DTLB, page table walker, satp, permission checks, and page fault generation.
39. `l1_i_cache.sv`
Implement per-core L1 instruction cache.
40. `l1_d_cache.sv`
Implement per-core L1 data cache.
41. `shared_l2_cache.sv`
Implement shared L2 cache.
42. `mesi_controller.sv`
Implement MESI cache coherence protocol.
43. `snoop_logic.sv`
Allow caches to observe coherence transactions from other cores.
44. `clint.sv`
Implement machine timer and software interrupts.
45. `plic.sv`
Implement platform-level external interrupt routing.
46. `interrupt_router.sv`
Route timer, software, and external interrupts to each hart.
47. `boot_rom.sv`
Provide reset code and boot entry into OpenSBI.
48. `uart.sv`
Provide serial output for Linux boot logs.
49. `dram_controller.sv`
Connect the CPU memory system to DRAM or a simulation DRAM model.
50. `ooo_core.sv`
Integrate frontend, decode, rename, dispatch, execution, memory, writeback, commit, CSR, trap, and MMU logic into one out-of-order core.
51. `multicore_riscv_cpu_top.sv`
Instantiate multiple out-of-order cores, shared memory system, cache coherence, interrupts, boot system, and external devices.

1.2 Final Success Criteria

The CPU is complete when it can:

- execute RV64GC binaries,
- run out-of-order execution,
- support multiple harts,
- support M/S/U privilege modes,
- support Sv39 virtual memory,
- support CLINT and PLIC interrupts,
- support atomic synchronization,
- maintain coherent shared memory,
- boot OpenSBI,
- boot Linux,
- run a BusyBox shell.

““